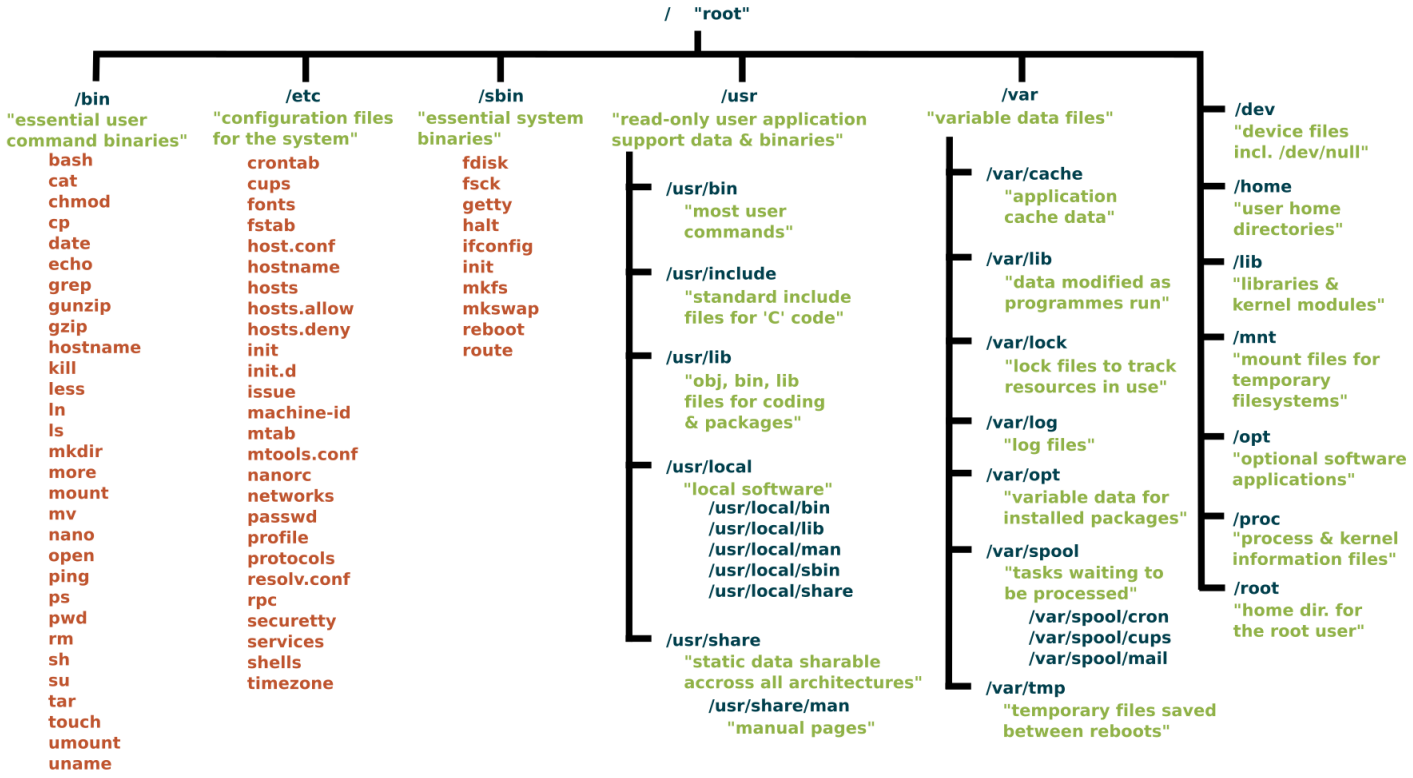
CORSO AVANZATO DI INFORMATICA E ROBOTICA

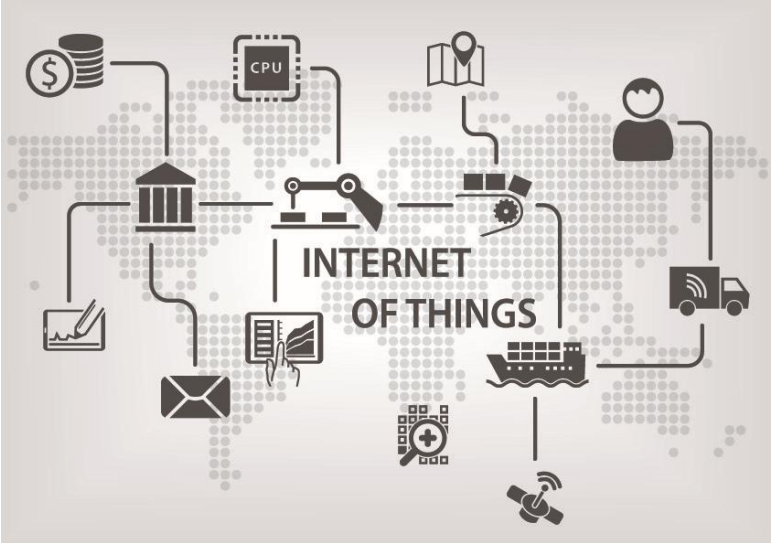
LEZIONE 11: Python

Carlo **Zambaldo** | 11 febbraio 2026

Punto della situazione

- Mancano 9 lezioni
- Gli ultimi argomenti da trattare sono:
  - **Script in Python**
    - Piccoli programmini di basic I/O
    - Librerie Python e pip
    - Accesso ai GPIO
    - cv2 – libreria di computer vision
  - **Accenni all'Intelligenza Artificiale**
    - Giochiamo con l'AI e il Raspberry!
      - LLM
      - Visione Artificiale (YOLO)
  - **Note conclusive del corso**
    - Dall'idea al progetto
    - … inclusi i vostri feedback!





Cosa rappresenta il «.py»? e il «.txt»?

Le estensioni dei file: cosa sono e perché contano

Un file ha sempre due parti: « *nome . estensione* »

- **Nome** → scelto dall'utente, per identificare il file
- **Estensione** → dice al computer *che tipo di file è*

Perché l'estensione è importante?

- Dice al computer **con quale programma aprire il file**
- **Identifica il tipo di dati contenuti nel file**
- Cambia **il comportamento del file**
- Senza estensione, il computer **non sa cosa farne**

Attenzione ⚠️

- Cambiare **solo il nome dell'estensione** NON cambia il tipo di file (foto.jpg → foto.txt)
- Windows spesso **nasconde le estensioni**
  → sembra che non esistano, ma ci sono! (basta attivarne la visibilità)

Categoria	Estensioni principali
Documenti	`.pdf`, `.docx`, `.txt`, `.rtf`
Immagini	`.jpg`, `.png`, `.gif`, `.svg`, `.tiff`
Video	`.mp4`, `.mov`, `.avi`, `.wmv`
Audio	`.mp3`, `.wav`, `.aac`, `.flac`
Archivi	`.zip`, `.rar`, `.7z`, `.tar.gz`
Eseguibili	`.exe`, `.msi`, `.bat`, `.sh`

Tavole di Verità e operatori logici

Operatori booleani

op.	descrizione
and	E' vero (true) se entrambi gli operandi sono veri. Altrimenti è falso (false).
or	E' vero (true) se almeno un operando è vero. Altrimenti è falso (false).
not	E' vero se l'operando è falso. E' falso se l'operando è vero.

Operatori binari

op.	descrizione
x << n	shift a sinistra di n posizioni bit di x
x >> n	shift a destra di n posizioni bit di x
x & y	and tra i bit di x e y
x | y	or tra i bit di x e y
x^y	or esclusivo tra i bit di x e y
~x	inverte i bit di x

p	$\overline{p}$
V	F
F	V

p	q	$p \wedge q$
V	V	V
V	F	F
F	V	F
F	F	F

p	q	$p \vee q$
V	V	V
V	F	V
F	V	V
F	F	V

p	q	$p \dot{\vee} q$
V	V	F
V	F	V
F	V	V
F	F	F

p	q	$p \rightarrow q$
V	V	V
V	F	F
F	V	V
F	F	V

p	q	$p \leftrightarrow q$
V	V	V
V	F	F
F	V	F
F	F	V

Operatore	Descrizione	espressione	Cosa restituisce
>	maggiore di	`x>y`	`true` se `x` è strettamente maggiore di `y`, `false` altrimenti
>=	maggiore o uguale di	`x>=y`	`true` se `x` è maggiore o uguale di `y`, `false` altrimenti
<	minore di	`x<y`	`true` se `x` è strettamente minore di `y`, `false` altrimenti
<=	minore o uguale di	`x<=y`	`true` se `x` è minore o uguale di `y`, `false` altrimenti
==	uguale	`x==y`	`true` se `x` è uguale a `y`, `false` altrimenti
!=	diverso	`x!=y`	`true` se `x` è diverso da `y`, `false` altrimenti

Python Fundamentals

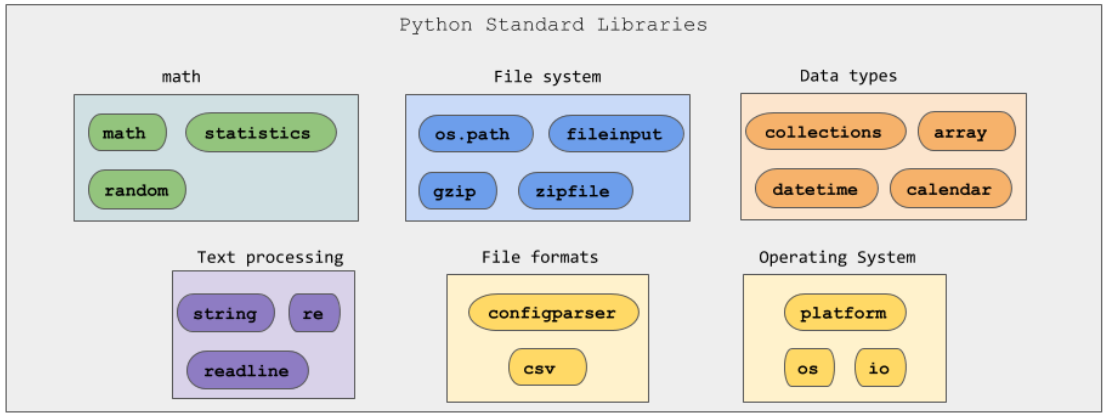
01

Python

Cos'è?

Python è un **linguaggio di programmazione**:

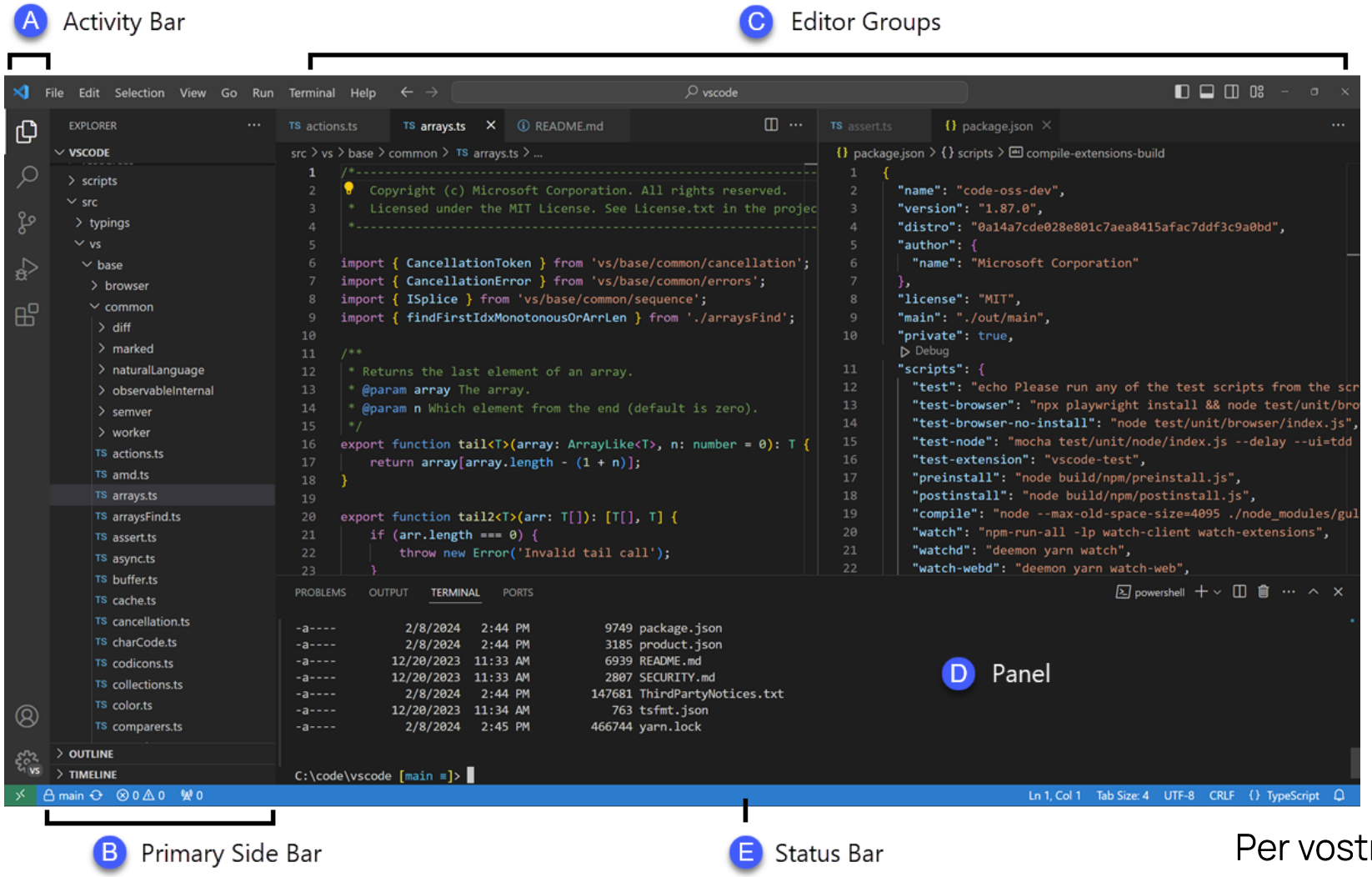
- semplice da leggere
- facile da imparare
- molto potente
- È uno dei linguaggi più usati al mondo.



Perché usiamo Python?

- Sintassi chiara (ad alto livello)
- Ideale per iniziare a programmare
- Funziona su Windows, Mac, Linux…
- Esistono numerosissime librerie che consentono applicazioni in:
  - Intelligenza Artificiale
  - Robotica
  - Automazione
  - Analisi dati
  - Web
  - …

Integrated Development Environment (IDE)



VS Code

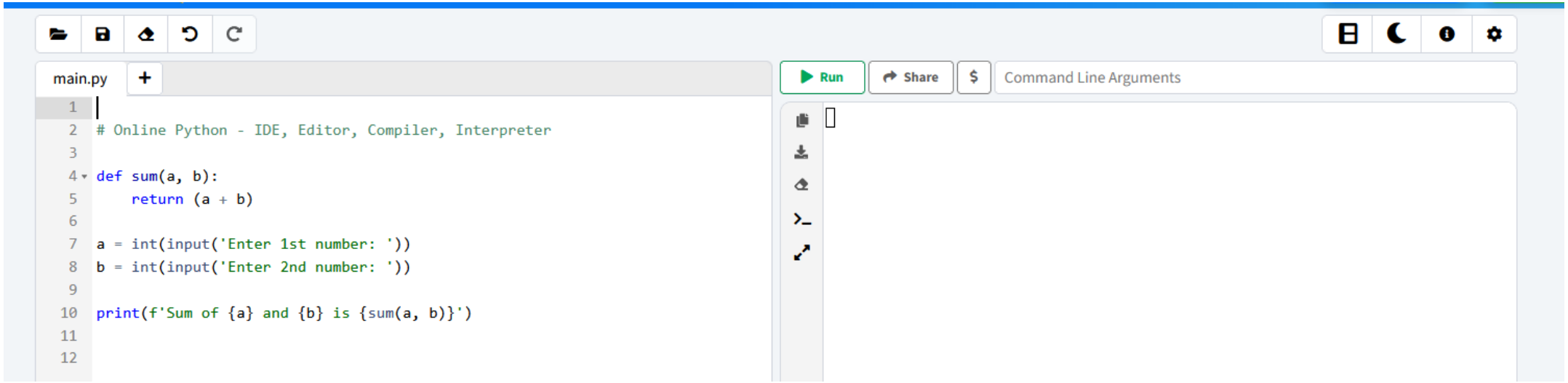
Per vostra convenienza potete usare VS code:
https://code.visualstudio.com/download

Online Python

- Chi di voi non avesse VS Code o stesse usando il PC della scuola:

https://www.online-python.com/

- Non sostituisce l'accesso al Raspberry!



Esercizio 3 – creiamo un file .py

- Aprire il file con il comando `nano`, seguito dal nome del file da creare
- Una volta aperto nano, puoi **scrivere normalmente** dalla tastiera:

```
print("Hello, Python!")
```

- Per salvare il file: Premi **CTRL + O** dunque premi **INVIO** per confermare il nome
- Provate a eseguire nuovamente il comando «ls» e noterete che il vostro file è apparso nella directory corrente

- Per eseguire il codice basterà chiamare l'interprete python con il comando `python3`, seguito dal nome del file appena generato, es.:

```
python3 test.py
```

- Nota: chiamare python3 senza nulla dopo apre l'interprete python!

```
>>> print("Hello World!")
Hello World!
>>>
```

while loop

Un while è un “finché”:
Finché questa condizione è vera, continua a fare qualcosa.

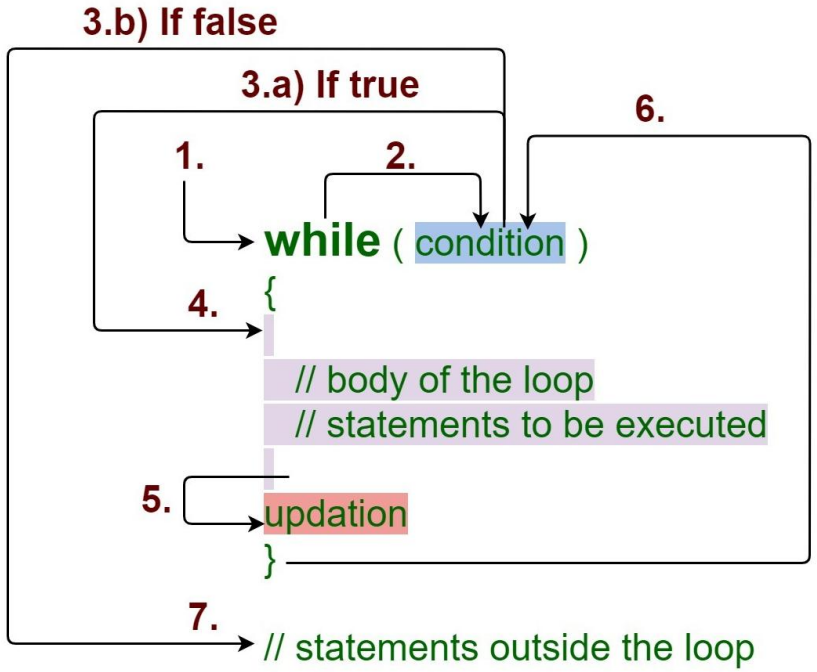
```
total = 0
number = int(input(" Please Enter any integer below 10:  "))
while (number <= 10):
    total = total + number
    number = number + 1
    print(" Value of Total : ", total)
else:
    print(" You Value is Greater Than 10 ==> This is from Else Block ")
```

IDLE Shell 3.9.2

```
Please Enter any integer below 10:  5
Value of Total :  5
Value of Total :  11
Value of Total :  18
Value of Total :  26
Value of Total :  35
Value of Total :  45
You Value is Greater Than 10 ==> This is from Else Block
```

tutorialgateway.org

While Loop



while loop – esempio

Un while è un "finché": *Finché questa condizione è vera, continua a fare qualcosa.*

- Esempio base: un contatore:

```
i = 0
while i < 5:
    print(i)
    i = i + 1
```

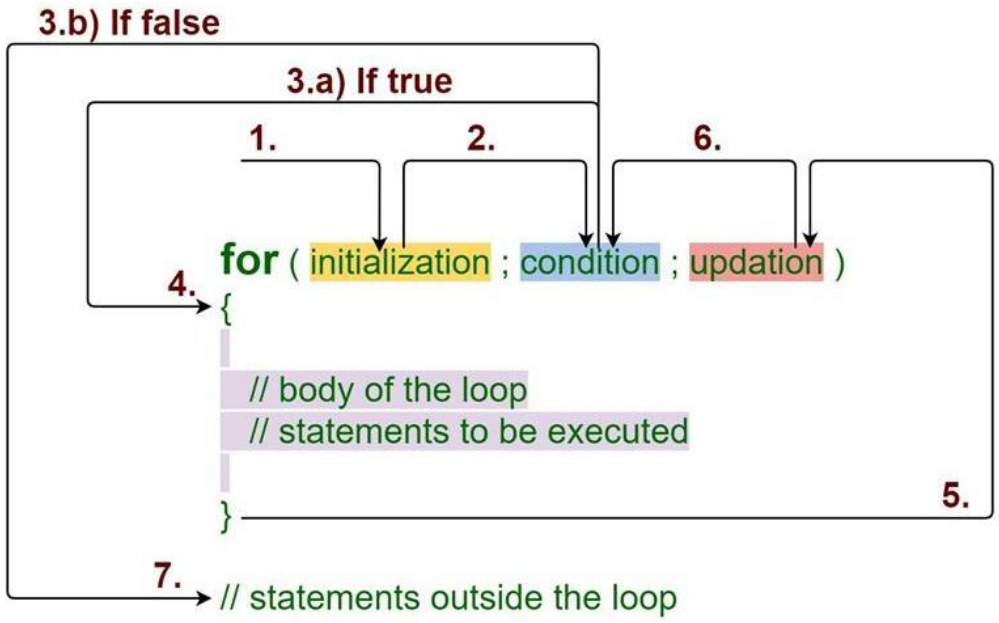
- Esempio «pratico»: verifica input utente

```
password = ""
while password != "ciao":
    password = input("Inserisci la password: ")

print("Accesso consentito")
```

for loop

Un for è un "per ogni":
Per ogni valore/elemento/numero fai …



Python for loop

A for loop is **used for iterating over a sequence and iterables** (like range, list, a tuple, a dictionary, a set, or a string).

```
for i in range(5):
        statement 1
        statement 2
        …
        statement n
else:
        statement(s)
```

Definite iterations.
(Total 5 iterations)

Indentation
Loop body is must be properly indented

Body of for loop
Execute till the last item of a sequence

Else Block (optional)
Execute only when for loop executes normally

for loop - esempio

Un for è un "per ogni": *Per ogni valore/elemento/numero fai ...*

- Esempio: un contatore

```
for i in range(5):
    print(i)
```

- Esempio: iterare una lista

```
nomi = ["Anna", "Luca", "Marco"]
for nome in nomi:
    print(nome)
```

if … else … statements

1st Condition is True

```
let number = 5
if number > 0 :
    # code

elif number < 0 :
    # code

else :
    # code


# code after if
```

2nd Condition is True

```
let number = -5
if number > 0 :
    # code

elif number < 0 :
    # code

else :
    # code


# code after if
```

All Conditions are False

```
let number = 0
if number > 0 :
    # code

elif number < 0 :
    # code

else :
    # code


# code after if
```

if … else … - esempi

==	uguale
!=	diverso
<	minore
<=	minore o uguale
>	maggiore
>=	maggiore o uguale

- Esempio semplice

```
temperatura = 25

if temperatura > 30:
    print("Fa caldo")
else:
    print("Temperatura ok")
```

- if … elif … else !

```
voto = 7

if voto >= 9:
    print("Ottimo")
elif voto >= 6:
    print("Sufficiente")
else:
    print("Insufficiente")
```

Gestione I/O in Python

02

Esempio – inserire dati da command window

La funzione input() permette di leggere dati inseriti dall'utente da tastiera.

- Il programma si ferma
- Aspetta che l'utente scriva qualcosa
- Quando premi INVIO, il valore viene salvato nella variabile

```
>>> a = input()
10
>>> b = input()
22
>>> a+b
'1022'
>>> |
```

```
num = input("inserire un numero intero: ")
print(f"Hai inserito: {num}")
```

input() restituisce sempre una **stringa** (testo)!!

```
>>> int(a)+int(b)
32
>>> |
```

Convertire i dati

```
eta = int(input("Quanti anni hai? "))
anno_prossimo = eta + 1
print("L'anno prossimo avrai", anno_prossimo)
```

Conversioni più comuni:

int() → numero intero

float() → numero con virgola

str() → testo

```
numero = float(input("Inserisci un numero: "))
if numero > 0:
        print("Numero positivo")
else:
        print("Numero negativo o zero")
```

Gestione File

Perché leggere un file?

- Salvare dati in modo permanente
- Riutilizzare informazioni (configurazioni, testi, numeri)
- Comunicare tra programmi

Per aprire un file:

```
file = open("dati.txt", "r")
```

nota: la «r» sta per read («a» = append, «w» = write)

Per leggere il contenuto:

```
contenuto = file.read()
print(contenuto)
```

Per chiudere il file

```
file.close()
```

È consigliato usare «with» per aprire il file, perché:

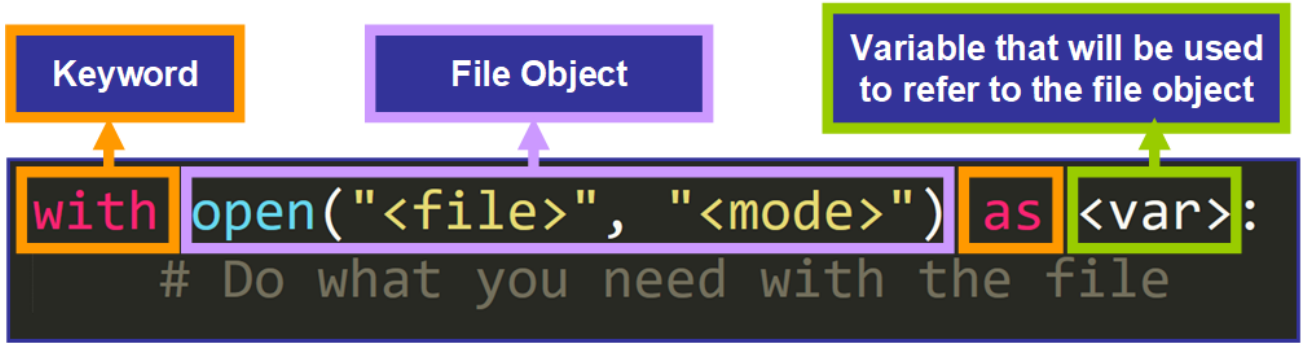
- il file si chiude automaticamente
- codice più pulito
- meno errori

```
with open("dati.txt", "r") as file:
    contenuto = file.read()
    print(contenuto)
```

Gestione File

Perché leggere un file?

- Salvare dati in modo permanente
- Riutilizzare informazioni
- Comunicare tra programmi



Le modalità disponibili sono:

- **Lettura ("r", read)**
- **Aggiunta ("a", append)**
- **Scrittura ("w", write)**
- **Creazione ("x", create)**

Puoi anche scegliere di aprire un file in:

- **Modalità di testo ("t")**
- **Modalità binaria ("b")**

https://www.freecodecamp.org/italian/news/i-file-in-python-aprire-leggere-scrivere-file-e-altre-funzioni-di-gestione-file/

È consigliato usare «with» per aprire il file, perché:

- il file si chiude automaticamente
- codice più pulito
- meno errori

```
with open("dati.txt", "r") as file:
        contenuto = file.read()
        print(contenuto)
```

Esercizio

1. Creare un file .txt usando «nano», scrivere quindi almeno due righe di testo
2. Aprire un file .py, scrivere il codice:

```
with open("dati.txt", "r") as file:
    contenuto = file.read()
print(contenuto)
```

3. Ora, provare ad eseguire
4. Cosa cambia se il codice è:

```
with open("dati.txt", "r") as file:
    for riga in file:
        print(riga, end="")
```

Scrivere su file

fondiamo insieme basic I/O con scrittura su file!

```
messaggio = input("Scrivi qualcosa per il diario: ")

with open("diario.txt", "a") as file:
    file.write(messaggio + "\n")

print("\nContenuto del diario:\n")

with open("diario.txt", "r") as file:
    for riga in file:
            print(riga, end="")
```

```
Scrivi qualcosa per il diario: ciaoooo

Contenuto del diario:

ciaoooo


** Process exited - Return Code: 0 **
```

Librerie Python e Ambienti Virtuali

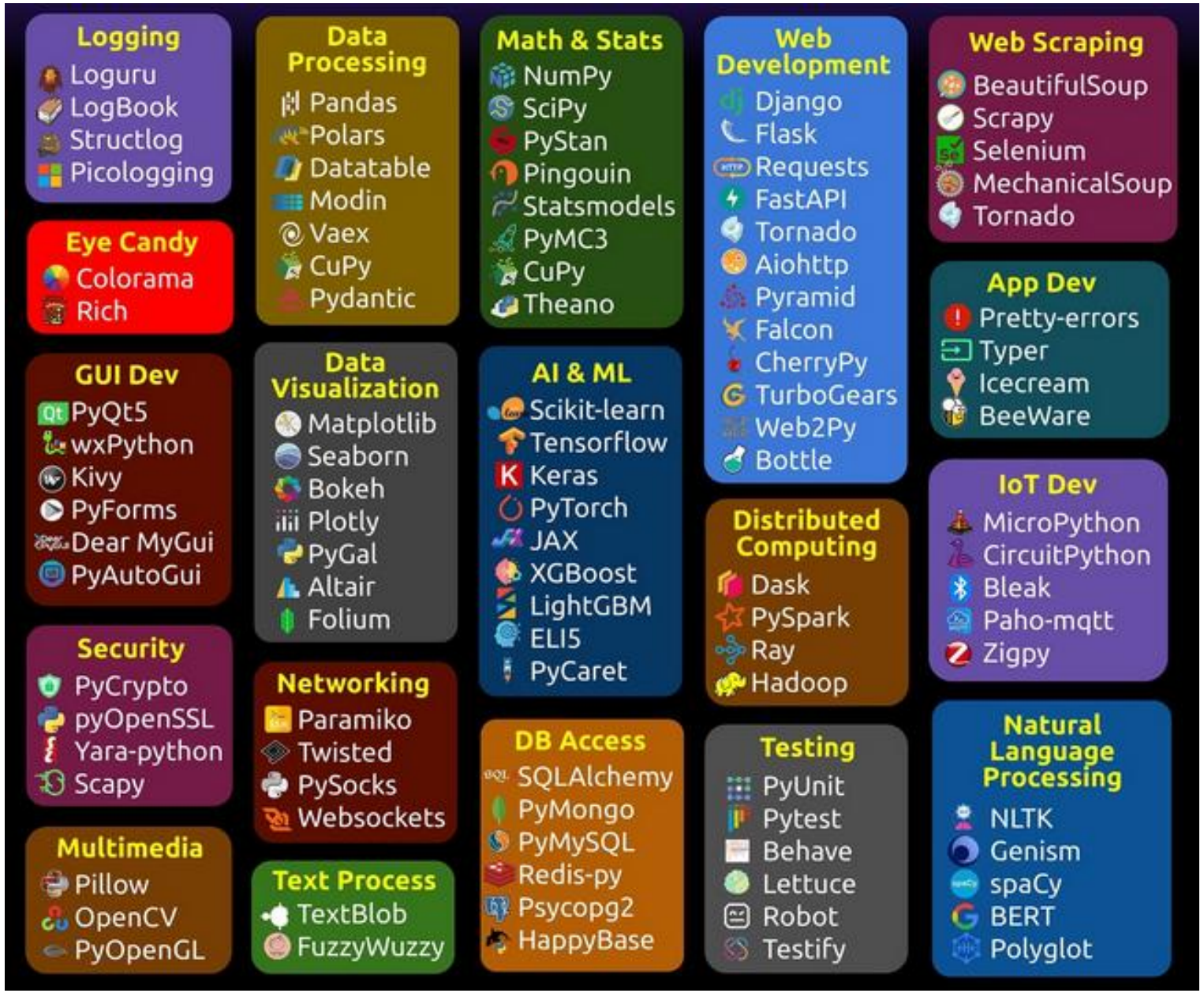
03

Librerie Python di terze parti

Uno dei principali punti di forza del linguaggio Python è il suo vasto ecosistema di librerie.

Sebbene le librerie integrate in Python siano già molto versatili, esistono moltissime librerie e framework di terze parti che rendono la programmazione in Python ancora più produttiva.

Ecco alcune delle librerie e dei framework open-source Python più utili e popolari.



Come importare una libreria / modulo

- **Import semplice**

```
import math
print(math.sqrt(16))
```

- **Import con alias**

```
import math as m
print(m.sqrt(25))
```

- **Importare solo una funzione**

```
from math import sqrt
print(sqrt(36))
```

Scrittura	Quando usarla
`import modulo`	più chiaro
`import modulo as x`	abbreviare
`from modulo import funzione`	usare poche funzioni

“Moduli”, “pacchetti”, “librerie”: cosa stai installando davvero?

- **Modulo**: un singolo file .py (es. mio_modulo.py).
- **Pacchetto**: una cartella con codice Python (spesso con __init__.py) e magari sotto-moduli.
- **Libreria**: termine generico (spesso coincide con un pacchetto installabile).

`pip` è il **package manager ufficiale di Python**.

Un *package manager* è un programma che:

- scarica librerie
- le installa
- gestisce le versioni
- risolve le dipendenze

```
pip install <nome_pacchetto>
```

Cosa fa pip:

1. si collega a **PyPI (Python Package Index)**
2. scarica il pacchetto **`<nome_pacchetto>`**
3. scarica automaticamente le sue dipendenze
4. installa tutto dentro il tuo ambiente Python

https://pypi.org/

Dove viene installato un modulo?

Il problema: su un computer possono esistere **più versioni di Python**:

- Python di sistema
- Python installato manualmente
- Python dentro un venv

Usando `pip install nome_pacchetto` non è sempre chiaro **quale Python** sta ricevendo il pacchetto, e si possono ottenere errori tipo: ModuleNotFoundError

È conveniente usare: `python3 -m pip install nome_pacchetto`
Questo dice al computer: «Usa pip appartenente a questo Python.»

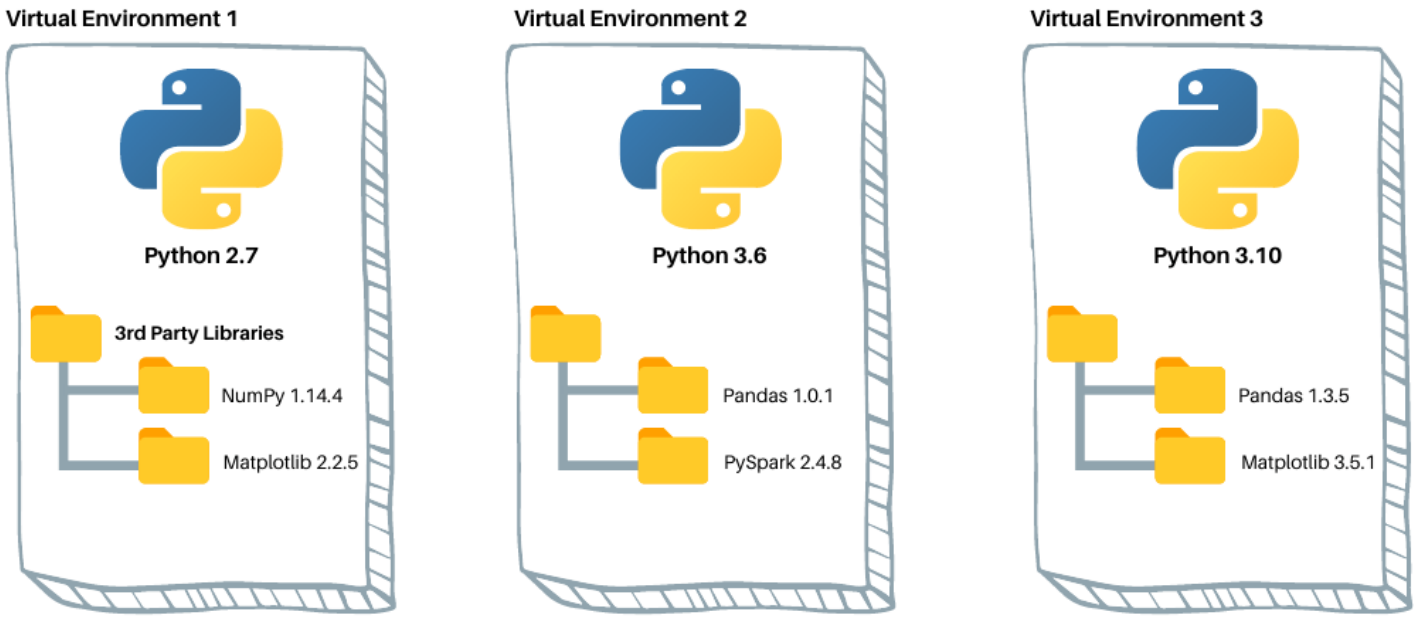
Il problema non è installare un modulo.
Il problema è installarlo nel Python giusto.

Un metodo ancora più sicuro è l'utilizzo degli ambienti virtuali

Ambienti Virtuali
Perché usare venv è conveniente?

Se installi tutto nel Python di sistema:

- Versioni che entrano in conflitto
- Progetti che smettono di funzionare
- Ambiente “sporco”
- Difficile replicare su un altro PC



venv crea un ambiente Python isolato.
Ogni progetto ha:

- **il suo Python**
- **le sue librerie**
- **le sue versioni**

Con installazione globale → possibili conflitti!

Benefici:

- **Nessun conflitto tra progetti**
- **Ambiente replicabile** (`pip freeze > requirements.txt` → ricrearli altrove con `pip install -r requirements.txt`)

Accenni sui Container

venv	Container (es. Docker / Podman)
Isola **solo Python e le sue librerie**	Isola: • Python • Librerie • Sistema operativo • Dipendenze di sistema • Configurazione
✔ Versioni di pacchetti indipendenti ✔ Leggero ✔ Facile da usare ❌ Non isola il sistema operativo ❌ Non isola librerie di sistema Funziona **dentro il tuo computer**.	È una **mini-macchina virtuale leggera**. ✔ Ambiente completamente replicabile ✔ Funziona uguale su ogni macchina ✔ Ideale per server / produzione ❌ Più complesso ❌ Più pesante di un venv

Domande, Dubbi, Perplessità?

```
from gpiozero import LED, Button
from time import sleep

led = LED(17)
button = Button(2)

while True:
    button.wait_for_press()
    led.toggle()
    sleep(0.5)
```